

SQL | SEQUENCES

SQL sequences are an essential feature of **relational database management systems** (RDBMS) used to generate **unique numeric values** in a sequential order. These values are widely used for generating **primary keys**, **unique keys**, and other numeric identifiers in **databases**. SQL sequences offer flexibility, performance, and ease of use, making them indispensable in managing and organizing data in **large-scale applications**.

In this article, we will explain SQL sequences in-depth, explaining how they work, their advantages, syntax, and real-world use cases.

What Are SQL Sequences?

SQL sequences are user-defined **database objects** designed to generate a series of **numeric values**. Unlike identity columns, which are tightly bound to specific tables, sequences are **independent objects** and can be used across multiple tables. They allow applications to retrieve the next number in a sequence whenever needed, offering a simple and efficient way to generate unique numbers on demand.

The values in a sequence can be configured to be generated in **ascending** or **descending order**, and the sequence can be set to **restart** (cycle) once the maximum value is exceeded. This makes **SQL sequences** particularly useful in scenarios where there is a need for continuous, unique values, such as generating primary keys or serial numbers.

Key Features of SQL Sequences

- **Automatic Primary Key Generation:** Sequences automatically generate unique values that can be used for primary or unique keys in database tables.
- **Ascending or Descending Order:** Sequences can be configured to generate numbers in either ascending or descending order.
- **Multiple Table Usage:** A single sequence can be used to generate values for multiple tables, making it flexible and reusable.
- **Independent of Tables:** Unlike identity columns, sequences are independent and can be used across different tables.
- **Efficient:** Sequences reduce the complexity and overhead of manually generating unique values, which saves time and reduces application code.

How SQL Sequences Work

When creating a sequence, we specify the **starting point**, the **increment** (how much the sequence increases with each step), and optionally the minimum and maximum values. Sequences can be set to cycle, which means they restart from the beginning when they reach the **maximum value**.

Syntax:

```
CREATE SEQUENCE sequence_name
START WITH initial_value
INCREMENT BY increment_value
MINVALUE minimum_value
MAXVALUE maximum_value
CYCLE|NOCYCLE ;
```

Example 1: Creating a Sequence in Ascending Order

```
CREATE SEQUENCE sequence_1
start with 1
increment by 1
minvalue 0
maxvalue 100
cycle;
```

Explanation:

The above query will create a sequence named **sequence_1**. The sequence will start from 1 and will be incremented by 1 having maximum value of 100. The sequence will repeat itself from the start value after exceeding 100.

Example 2: Creating a Sequence in Descending Order

```
CREATE SEQUENCE sequence_2
start with 100
increment by -1
min value 1
max value 100
cycle;
```

Explanation:

The above query will create a sequence named **sequence_2**. The sequence will start from 100 and should be less than or equal to a maximum value and will be incremented by -1 having a minimum value of 1.

Using SQL Sequences in Database Operations

Once a sequence is created, it can be used across **multiple tables** to generate **unique values**, such as primary key identifiers or **serial numbers**. This allows for **consistent**, efficient value generation, reducing the need for **manual input** and ensuring **uniqueness** across different rows and tables.

Example: Using a Sequence to Insert Values

Let's create a **students** table and use the sequence to automatically generate unique student IDs. In this example, the [NEXTVAL](#) function is used to retrieve the next value in the sequence (sequence_1) and insert it into the ID column for each student.

Query:

```
CREATE TABLE students
(
ID number(10),
NAME char(20)
);

INSERT into students VALUES
(sequence_1.nextval,'Shubham');
INSERT into students VALUES
(sequence_1.nextval,'Aman');
```

Output

ID	NAME
1	Shubham
2	Aman

Using Sequence to Insert Values

Cache Management in SQL Sequences

To enhance performance, SQL Server employs [cache management](#) for sequence numbers. The CACHE argument pre-allocates a set number of sequence values in memory, which reduces disk access during normal operations.

Example of Cache Management

For instance, if a sequence starts at 1 with a **cache size of 15**, SQL Server will allocate values 1-15 in memory. When the 15th value is used, a new cache with values **16-30** is allocated.

In the event of a server restart, the next sequence number will be the first unused value in the cache. This approach ensures that sequence numbers are generated quickly but may cause gaps in the sequence after a **server crash**, which is normal.

If we want to avoid gaps, we can set the sequence to **NOCACHE**, though this might reduce performance as it requires constant disk access.

Creating a Sequence with Cache

```
CREATE SEQUENCE sequence_3  
START WITH 1  
INCREMENT BY 1  
CACHE 10;
```

Practical Use Cases for SQL Sequences

1. Primary Key Generation

Sequences are commonly used to generate **unique primary key** values for database tables. This is especially useful in applications where a large volume of records needs to be inserted into a table, and each record requires a unique identifier.

2. Serial Numbers and Order IDs

SQL sequences can be used to generate **serial numbers** for products or order IDs in **e-commerce systems**, ensuring that each order has a unique identifier.

3. Auditing and Tracking

Sequences are also useful for **tracking events** or **transactions** that require unique identifiers, such as logging system activities or generating unique reference numbers for transactions.

Advantages of Using SQL Sequences

- **Efficiency:** Sequences simplify the process of generating unique values without requiring complex application logic.
- **Independence:** Sequences can be used across multiple tables, ensuring that unique values are consistently generated without tying them to a specific table.
- **Performance:** By using the [CACHE](#) feature, sequences can enhance performance by reducing disk access during value generation.
- **Flexibility:** SQL sequences can be configured to generate values in ascending or descending order and can be set to cycle when reaching the maximum value.

Conclusion

SQL sequences are a fundamental tool for generating unique **numeric values** in databases, offering **simplicity**, **flexibility**, and improved performance. Whether we need to generate **primary keys**, serial numbers, or unique transaction IDs, sequences provide a reliable and efficient solution. Understanding how to create and manage sequences, along with optimizing **cache settings**, can significantly enhance the performance of your [database](#) operations.